Attention 加速器 IP 设计与验证报告

数字组招生实践项目（大二）· A 题 · 规模 S = 8、d = 4 · SystemVerilog + Icarus Verilog 12.0 · 2026 年 10 月

本报告记录基础 Attention 加速器 IP 的完整实现过程：从题目公式出发完成定点方案与位宽推导，用 SystemVerilog 实现可综合、可仿真的 RTL，再用 Python 黄金模型和自检测试平台完成逐位验证。当前版本单次任务耗时 169 个时钟周期，在 100 MHz 下约 1.69 μs；41 个定向与随机用例、409 个压力用例全部通过，与黄金模型逐位一致。报告同时给出资源估计、定点精度分析和序列长度扩展后的影响评估。

1 任务理解与实现范围

题目要求围绕 Q、K、V、Score、Softmax 和加权求和，实现一个可综合、可仿真的基础 Attention RTL IP：规模固定为 S = 8 个 Token、d = 4 维，输入输出为 8-bit 有符号整数；必须先用最大值单元求 x_max，再计算 exp(score - x_max)；weight 为无符号定点数、小数固定保留 4 位、整数位宽自行推导；最终输出 O 的每个元素由 round(ACC / L) 得到。

本设计覆盖全部基础验收项，并额外完成了参数化、波形自动渲染与精度分析。表 1 对照验收项说明实现情况。

表 1 基础验收项与实现情况对照

验收项	题目要求	本设计实现情况
编译	RTL 与 testbench 可在指定仿真器中编译	Icarus Verilog 12.0，-g2012 -Wall 编译通过
功能	通过公开样例与隐藏随机样例	41 用例与 409 压力用例全部通过，与黄金模型逐位一致
时序	reset、start、valid、ready、done 符合约定	见第 6 节；testbench 检查 done 时序与输出 X 值
结构	必须实现 x_max 与 exp(score - x_max)，位宽合理	最大值比较单元 + exp_lut 查找表，位宽推导见第 5 节
可解释性	现场说明数据流与关键代码	第 3、4 节给出数据流、状态机与逐拍波形

2 算法理解与手算核对

一次 Attention 的计算可以拆成三步：用 Query 与每个 Key 做点积得到相关性分数 score_j；用 Softmax 把分数变成非负权重；再用权重对 Value 加权求和并归一化。写成公式为：

```
score_j = Σ_k Q[k] × K_j[k]        (j = 0 … S-1)
x_max   = max(score_j)
weight_j = round( exp(score_j - x_max) × 2^4 )   // 无符号定点，小数 4 位
L = Σ_j weight_j        ACC[k] = Σ_j weight_j × V_j[k]        O[k] = round( ACC[k] / L )
```

为保证与题目规则完全一致，先用题目中的手算示例（S = 2、d = 2）核对定点规则：Q = [1, 2]，K0 = [1, 0]，K1 = [0, 1]，V0 = [4, 2]，V1 = [1, 6]。计算得 score = [1, 2]，x_max = 2；按 4 位小数规则 weight = [6, 16]（分别代表 0.375 与 1.000），L = 22，ACC = [40, 108]，O = [2, 5]，与题目结果一致。

这一步确认了两个容易出错的细节：指数输入必须先减去 x_max，使输入不超过 0、避免溢出；weight 是先把 exp 值缩放到定点再取整，而不是对 exp 值取整。

3 硬件架构与数据流

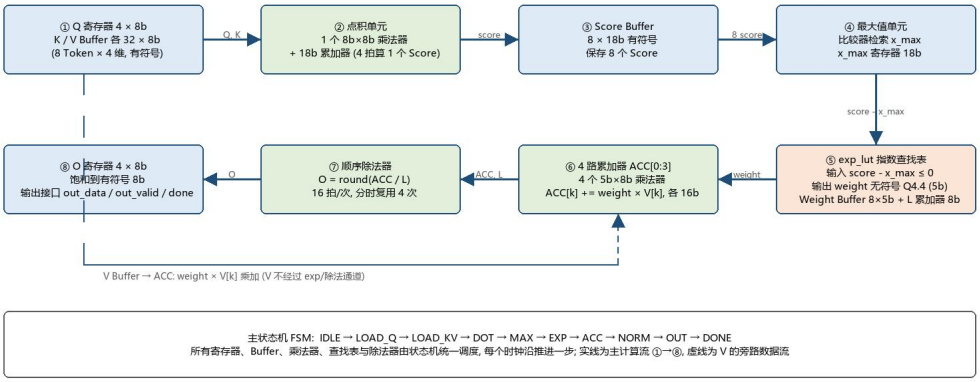


图 1 Attention IP 数据通路与存储结构

设计拆成三个模块：attention_top 负责状态机调度与全部数据通路，exp_lut 实现可综合的指数查找表，seq_div 实现带四舍五入的顺序除法器。RTL 与 testbench 均使用 SystemVerilog（IEEE 1800-2012）：状态机用 typedef enum 描述 11 个状态，时序逻辑用 always_ff、组合逻辑用 always_comb，位宽用 int 参数与 $clog2 推导。表 2 给出模块职责与关键实现。

表 2 模块划分与职责

模块	主要功能	关键实现
attention_top	装载、点积、检索 x_max、加权累加、归一化与输出调度	11 个状态的枚举状态机（always_ff）；Q 寄存器、K/V Buffer、Score Buffer、Weight Buffer、4 路 ACC
exp_lut	实现 weight = round(exp(score - x_max) × 16)	5 项查找表（case 结构），输入 score - x_max，输出无符号 Q4.4
seq_div	计算 O = round(ACC / L)	恢复余数法顺序除法器，16 拍完成一次，分时复用 4 次

数据在片上按“从输入到输出”的顺序流动：Q 与 K/V 先进入寄存器与 Buffer；点积单元每次从 Buffer 读取一个 K 元素与 Q 相乘并累加，4 拍得到一个 Score；8 个 Score 全部写入 Score Buffer 后检索最大值，再逐个查表得到权重并累加 L；随后用权重与 V 做 4 路并行乘加；最后由顺序除法器分时完成 4 次 ACC/L 并依次输出。

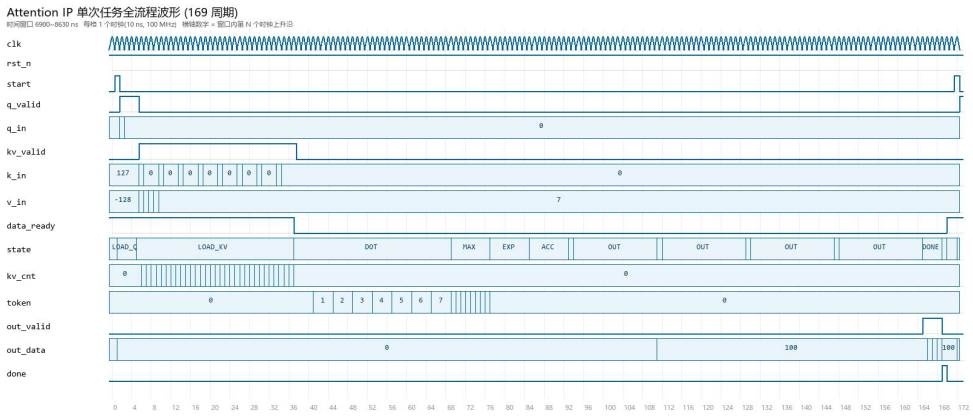


图 2 单次任务全流程波形（169 个时钟周期）

图 2 是单次任务的完整仿真波形：start 之后依次出现 Q 装载、K/V 装载、点积、最大值检索、指数查表、加权累加、归一化与输出，最后 done 拉高 1 拍回到空闲。

4 关键模块设计与舍入处理

4.1 点积单元与 Score 存储

点积使用单个 8-bit × 8-bit 有符号乘法器串行完成：每拍从 K/V Buffer 取出一个元素与对应的 Q 元素相乘，累加到 18 位累加器，4 拍得到一个 Score，8 个 Score 依次写入 Score Buffer。选择串行结构是为了用最小的乘法器面积完成计算，代价是 32 拍的点积时间。图 3 显示 token 计数器按 0 到 7 推进，每个 Token 用 4 拍完成 4 次乘加，累加器的中间值随乘法结果逐步变化。

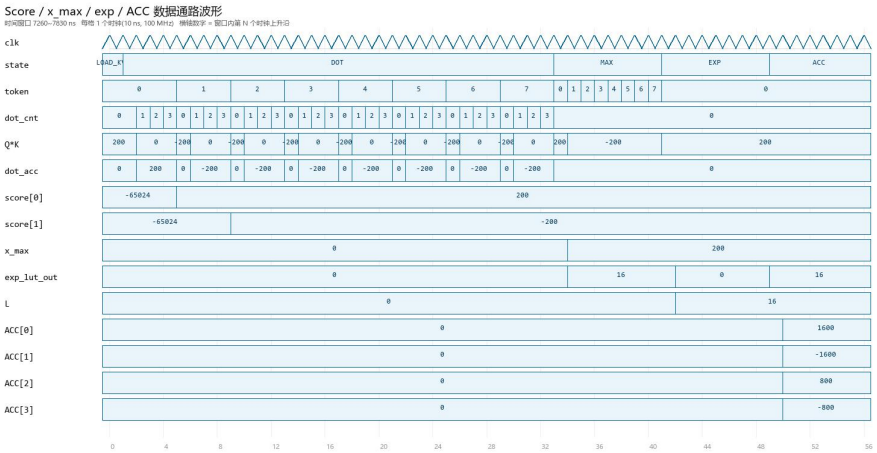


图 3 点积、x_max 检索、加权累加的数据通路波形

4.2 x_max 检索

最大值检索同样采用串行方式：先取 Score[0] 作为初值，随后每拍与下一个 Score 比较，8 拍完成 8 个 Score 的检索，只需一个 18 位比较器。图 3 中可以看到 x_max 在 MAX 阶段被更新为最大值，随后 EXP 阶段使用该值计算 score - x_max。

4.3 指数查找表与权重量化

权重按题目要求保留 4 位小数，因此指数的取值范围是 (0, 1]，只需要 1 位整数位，共 5 位。由于 Score 是整数，score - x_max 也是整数，查找表只需要覆盖有限的几个地址，表 3 给出完整表内容。

表 3 exp_lut 查找表内容（小数 4 位，整数 1 位）

mag = x_max - score	0	1	2	3	4	≥ 5
exp(-mag)	1.0000	0.3679	0.1353	0.0498	0.0183	< 0.007
× 16 后取整 (weight)	16	6	2	1	0	0

mag 大于等于 5 时 exp 值乘以 16 后不足 0.5，按四舍五入自然为 0，因此表中只需要 5 个有效地址，综合后是一小段组合逻辑。这也说明 4 位小数精度下，分数相差 5 以上的 Token 权重会下溢为 0，这一点在第 7 节的精度分析中会量化说明。

4.4 加权累加

加权累加阶段每个 Token 用一拍：4 个乘法器同时计算 weight_j × V_j[k] 并累加到 4 个 16 位累加器 ACC[k]，8 拍完成全部 8 个 Token。由于 weight 最多为 16、V 的绝对值最多为 128，单个累加器的绝对值不超过 128 × 128 = 16384，16 位累加器不会溢出。

4.5 归一化与舍入

O = round(ACC / L) 由顺序除法器完成。实现时先取 ACC 的幅值，加上 L 的一半后做整除，再按 ACC 的符号恢复正负；除法器采用恢复余数法，每个被除数位一拍，一次 16 拍，4 个输出元素分时复用同一个除法器。

这一实现与“四舍五入、遇到 0.5 时远离 0”完全等价：当 L 为偶数时 L/2 恰好等于 0.5L；当 L 为奇数时，等式 2|ACC| = L(2n+1) 左边为偶数、右边为奇数，不可能成立，因此不会出现正好落在半整数上的平局。图 4 给出 4 次除法与输出时序：div_start 每 18 拍出现一次，div_done 与商同时有效，4 个结果写入 O 寄存器后在 out_valid 期间依次输出。

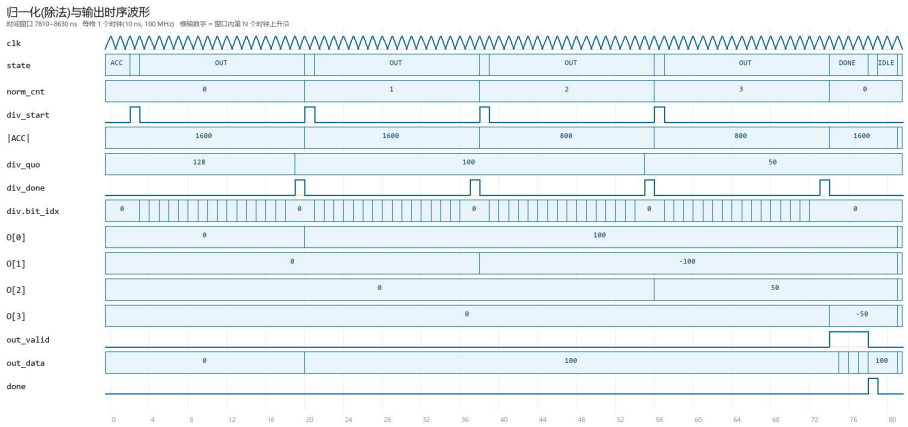


图 4 归一化除法与输出时序波形

5 位宽推导

位宽按“取值范围决定位宽”的原则逐级推导，保证既不截断也不浪费。表 4 汇总了各级数据的位宽与依据。

表 4 各级数据位宽与推导依据

数据	位宽	取值范围与依据

数据	位宽	取值范围与依据
Q / K / V / O	8-bit 有符号	题目规定，取值范围 -128 … 127
乘法结果 Q×K	16-bit 有符号	8b × 8b，范围 -16384 … 16384
Score / x_max	18-bit 有符号	4 项点积累加，范围 -65536 … 65536
score - x_max	19-bit 有符号	两个 18 位相减，且结果恒不大于 0，留 1 位余量
weight	5-bit 无符号（Q4.4）	exp 值在 (0, 1]，1 位整数 + 4 位小数，取整后 0 … 16
L = Σweight	8-bit 无符号	最多 8 × 16 = 128
ACC[k] / O[k]	ACC 16-bit 有符号、O 8-bit 有符号	|ACC| ≤ L × max|V| = 128 × 128 = 16384；除法商在输出前饱和

6 接口与时序约定

表 5 接口信号约定

信号	方向	约定
start	输入	仅在空闲状态有效，拉高 1 拍启动一次任务
q_in / q_valid	输入	q_valid 为 1 时 q_in 有效，共 4 拍，依次为 Q[0] … Q[3]
k_in / v_in / kv_valid	输入	kv_valid 为 1 时 k_in、v_in 同时有效，共 32 拍，第 j×4+k 拍给出 K_j[k] 与 V_j[k]
data_ready	输出	为 1 时允许发送输入；装载阶段为 1，计算阶段为 0
out_data / out_valid	输出	out_valid 为 1 时 out_data 有效，共 4 拍，依次为 O[0] … O[3]
done	输出	4 个结果输出完毕后拉高 1 拍，随后回到 IDLE

其中 clk 为同步时钟，rst_n 为低有效异步复位，复位后状态机回到 IDLE。题目接口中 k_in 与 v_in 都是 8-bit，而一个 Token 的 K_j 与 V_j 各有 d = 4 个元素，无法在一拍内传完一个 Token。因此本设计按“每拍给出

一个 K 元素和对应的一个 V 元素”组织输入，共 S × d = 32 拍，顺序为 K0[0]、V0[0]，K0[1]、V0[1]，…，K7[3]、V7[3]；testbench 按同样顺序驱动，波形中 kv_cnt 从 0 计数到 31。

7 验证方案与结果

验证采用“黄金模型 + 自动比对”的方式：Python 黄金模型严格按题目规则计算 Score、x_max、weight、L、ACC 与 O，并把期望结果写进激励文件；testbench 按接口协议驱动被测模块，采集 4 拍输出后逐项比对，同时检查 done 时序与输出是否出现 X 值。

表 6 测试用例与结果

测试类型	用例数	覆盖内容	结果
定向用例	9	全零、正负极值、单 Token 主导、等权重、负输出、舍入边界、题目示例构造	全部通过
随机用例	32	全量程 ±128 与小量程 ±8 各半，随机种子可配置	全部通过
压力测试	409	400 个随机用例（种子 12345）+ 9 个定向用例	全部通过

完整仿真日志保存在 sim/sim_log.txt，可以直接看到每一次任务的输出与周期数，日志截图见提交目录中的 doc/sim_log.png。

由于题目规定 weight 只保留 4 位小数，定点实现与全精度浮点 Softmax 会有差异。为量化这一影响，把 RTL 结果、定点黄金模型与浮点参考三者对比：RTL 与定点黄金模型在全部 409 个用例上逐位一致；与浮点参考相比，偏差随输入取值范围变化，结果见表 7。

表 7 4 位小数权重带来的量化偏差（每档 200 个随机用例）

输入取值范围	权重下溢为 0 的比例	与浮点参考不一致的用例	输出最大偏差
-128 … 127（全量程）	87.5%	0 / 200	0
-8 … 7（小量程）	85.2%	42 / 200	3
-2 … 1（极小量程）	48.7%	193 / 200	9

全量程输入下 Score 差异很大，Softmax 变得非常尖锐，只有一个 Token 的权重不为 0，此时量化不改变最终输出，因此与浮点参考完全一致；输入范围较小时权重分布更平坦，4 位小数的量化误差被放大，输出最多相差 9。可见当前实现完全满足题目规定的定点规则，但在“软” Softmax 场景下精度有限，第 10 节给出提高精度的可选方案。

8 性能与资源分析

8.1 周期数与吞吐

单次任务从 start 被采样到最后一个输出拍共 169 个时钟周期，其中装载 36 拍、点积 32 拍、检索最大值 8 拍、指数与权重 8 拍、加权累加 8 拍、归一化 72 拍、输出 4 拍。归一化占了 43% 的时间，原因是顺序除法器被 4 个输出元素分时复用。在 100 MHz 时钟下，单次任务延迟 1.69 μs；done 之后模块立即回到空闲，可以连续启动下一个 Query。

从计算效率看，一次任务共完成 32 次点积乘加与 32 次加权乘加，平均每拍约 0.38 次乘加，峰值出现在加权累加阶段的每拍 4 次乘加。

8.2 资源估计

表 8 资源估计

资源类别	数量与位宽	说明
乘法器	1 个 8b×8b + 4 个 5b×8b	点积串行复用 1 个；加权累加 4 路并行
比较器与累加器	1 个 18-bit 比较器；18-bit / 8-bit / 4×16-bit 累加器	x_max 检索、点积累加、L 累加与 4 路 ACC
除法器与查找表	1 个 16 拍顺序除法器、5 项 × 5-bit 查找表	除法器分时复用 4 次；查找表综合为组合逻辑
存储	824 bit（Q 32 + K 256 + V 256 + Score 144 + Weight 40 + ACC 64 + O 32）	片上寄存器/Buffer，不占用外部存储
状态与控制	11 状态状态机 + 8 个计数器（最宽 5 位）	q_cnt/kv_cnt/tok/dot_cnt/exp_cnt/acc_cnt/norm_cnt/out_cnt，统一调度各计算单元

8.3 序列长度扩展到 256 时的影响

题目要求讨论 S 从 8 扩展到 256 的变化。规模扩大 32 倍后，计算量、Buffer 容量和带宽需求同步增长，其中存储与访存的压力最突出，结果见表 9。

表 9 S = 8 与 S = 256 的计算量、存储与带宽对比

项目	S = 8（本设计）	S = 256	变化
点积乘加次数	8 × 4 = 32	256 × 4 = 1024	32 倍
加权乘加次数	8 × 4 = 32	256 × 4 = 1024	32 倍
片上 Buffer 容量	Score + Weight 184 bit，K/V 合计 512 bit	Score + Weight 5888 bit，K/V 合计 16 kbit	32 倍
片上存储合计（含 ACC/O）	824 bit	约 26 kbit	32 倍

当 S = 256 时，仅 K/V Buffer 就需要 2 KB，加上 Score 与 Weight 后总存储需求超过 3 KB，接近小型 FPGA 的片上 RAM 规模；若不能全部放在片上，每个 Query 都要重新搬运 2 KB 的 K/V，带宽会成为主要瓶颈。常见做法是对多个 Query 批量处理以复用 K/V、改用块状在线 Softmax 避免保存完整 Score 矩阵，或用压缩格式减少搬运量。

9 实现中遇到的问题与解决过程

累加计数器位宽不足导致的死循环是调试中最关键的问题。加权累加阶段需要按 Token 从 0 计数到 7，但计数器最初按维度位宽只分配了 2 位、最大只能计到 3，状态机在累加状态反复循环，仿真始终不给出 done。通过波形看到状态长时间停在 ACC 状态，再核对计数器取值范围定位到该错误，改为 3 位后 41 个用例一次性全部通过。

另外两处问题出在可观测性与数据残留上。Icarus Verilog 不把模块内数组写入 VCD，最初的波形看不到 Score、ACC 等 Buffer 内容，解决办法是在 testbench 中用普通 wire 镜像这些数组元素，仅用于波形观察，不影响 IP 接口。新任务开始时 ACC 与 L 会保留上一次任务的值，现在在采样到 start 的同一拍就把它们清零，既让波形清晰，也避免异常时序下旧数据影响输出。

10 已实现的加分项与改进方向

在完成基础功能之后，本设计实现并验证了题目第十二节中的三项加分项：A 与 B 属于“必须附独立测试”的功能项，C 属于参数化。加分功能全部由参数控制，基础版本默认参数不变，功能零回归（41 + 409 例仍全部通过、单次任务仍是 169 拍）。

表 10 已实现的加分项与实测结果

加分项	做法	实测结果
A 因果 Mask	新增 pos_i 输入与 MASK_EN 参数：j > pos_i 的 Key 不参与 x_max 检索、权重置 0、不进入 L 与 ACC	独立测试 tb_attention_mask.sv 检查权重 / L / 输出三层，pos_i = 0 / 3 / 7 三组全部通过
B 并行点积	新增 DOT_PAR 参数：D 个乘法器一拍算完一个 Score（点积 32 拍 → 8 拍）	周期 169 → 145 拍（−14%），41 例与 409 例压力测试全部通过；代价是 +3 个 8×8 乘法器与 3 个 18 位加法器
C 参数化	S / D / DW / W_FRAC 全部参数化，计数器位宽由 $clog2 自动推导	S 改为 4 后激励自动适配：41/41 通过，周期 169 → 125 拍

仍可继续优化：用倒数 LUT 或并行除法器把归一化从 72 拍降到 4 至 20 拍；把 weight 提升到 Q1.15 或用分段线性 / CORDIC 实现指数，以减小量化偏差；此外 D 项（动态阈值筛选）与 E、G 两项调研可在此基础上继续展开。

11 结论

本设计完成了题目要求的基础 Attention RTL IP：11 状态状态机调度装载、点积、x_max 检索、指数查表、加权累加、归一化与输出，单次任务 169 拍、片上存储 824 bit，450 个用例（41 + 409）与黄金模型逐位一致；并实现了加分项 A（因果 Mask）、B（并行点积，169 → 145 拍）与 C（参数化）。不足是 4 位小数权重在软 Softmax 场景下量化误差最大到 9，可用更高精度的指数实现进一步改善。